AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm drzewa BST
z zastosowaniem GitHub

Autor:
Piotr Majchrzak
Kamil Kwiczal

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **5**

1.1. Wprowadzenie do projektu 5
1.2. Cel projektu 5
1.3. Przeznaczenie aplikacji 6
1.4. Binarne drzewo poszukiwań BST 7
1.4.1. Zasada działania drzewa BST 8
1.4.2. Właściwości i wydajność BST 9
1.5. Sposoby przechodzenia drzewa 9
1.5.1. Preorder 10
1.5.2. Inorder 10
1.5.3. Postorder 11
1.5.4. Porównanie metod przechodzenia drzewa 12
1.6. Wykorzystanie kontenera STL std::vector 13
1.7. Wymagane technologie i narzędzia 14
1.7.1. Język C++ 15
1.7.2. Git i GitHub 15
1.8. Podsumowanie rozdziału 16

2. Analiza problemu **17**

2.1. Obszary zastosowania drzew BST 17
2.2. Analiza problemu 17
2.3. Analiza sposobu działania programu 18
2.4. Analiza poszczególnych operacji 19
2.4.1. Dodawanie elementu 20
2.4.2. Wyszukiwanie elementu 20
2.4.3. Usuwanie elementu 20
2.4.4. Czyszczenie drzewa 21
2.4.5. Wyświetlanie struktury 21
2.5. Praktyczny przykład działania algorytmu 21
2.6. Analiza zapisu i odczytu danych 23

2.7. Podsumowanie analizy 23

3. Projektowanie **24**

3.1. Założenia projektowe 24
3.2. Struktura plików projektu 24
3.3. Charakterystyka plików 25
3.4. Projekt klas 25
3.4.1. Klasa Node 25
3.4.2. Klasa BST 26
3.4.3. Klasa FileManager 26
3.4.4. Plik main.cpp 26
3.5. Diagram klas i zależności 27
3.6. Projekt przepływu działania aplikacji 27
3.7. Zarządzanie pamięcią 28
3.8. Projekt obsługi plików 29
3.8.1. Format tekstowy TXT 29
3.8.2. Format binarny BIN 29
3.9. Projekt procesu kompilacji 29
3.10. Organizacja pracy nad projektem 30
3.11. Praca zespołowa z wykorzystaniem Git i GitHub 31
3.11.1. Commity i gałęzie 31
3.11.2. Scalanie zmian i rozwiązywanie konfliktów 31
3.11.3. Weryfikacja scalonego kodu 32
3.11.4. Dokumentacja pracy w GitHub 33
3.12. Podsumowanie projektowania 36

4. Implementacja **38**

4.1. Implementacja struktury drzewa 38
4.2. Implementacja operacji na drzewie BST 38
4.2.1. Dodawanie elementów 38
4.2.2. Usuwanie elementów 39
4.2.3. Czyszczenie drzewa 40
4.2.4. Wyszukiwanie drogi do elementu 41

4.2.5. Wyświetlanie drzewa 42
4.3. Implementacja obsługi plików 42
4.3.1. Zapis i odczyt plików tekstowych 42
4.3.2. Zapis i odczyt plików binarnych 42
4.4. Implementacja menu programu 44
4.5. Testowanie programu i uzyskane wyniki 45
4.6. Dokumentacja kodu za pomocą Doxygen 46
4.7. Podsumowanie implementacji 47

5. Wnioski **48**

Spis rysunków **50**

Spis tabel **51**

Spis listingów **52**

Literatura **53**

1. Ogólne określenie wymagań

1.1. Wprowadzenie do projektu

Celem projektu jest zaprojektowanie i wykonanie aplikacji konsolowej w języku C++, której zadaniem jest tworzenie oraz obsługa binarnego drzewa poszukiwań (BST – Binary Search Tree) [5]. Program umożliwia przechowywanie wartości liczbowych w uporządkowanej strukturze danych, a następnie wykonywanie na nich podstawowych operacji, takich jak dodawanie, usuwanie i wyszukiwanie elementów.

Drzewa binarne są jednym z podstawowych zagadnień wykorzystywanych w informatyce [3]. Pozwalają na organizowanie danych w sposób hierarchiczny, co może znacząco usprawnić ich wyszukiwanie oraz modyfikowanie. W przeciwieństwie do zwykłej tablicy lub listy, elementy drzewa nie są przechowywane w jednym ciągu [8]. Każdy węzeł może wskazywać na kolejne elementy, tworząc strukturę przypominającą odwrócone drzewo.

Projekt ma charakter edukacyjny i praktyczny. Jego realizacja pozwala na poznanie zasad działania dynamicznych struktur danych, wykorzystania wskaźników w języku C++ [1], rekurencji oraz obsługi plików. Dodatkowo praca nad projektem umożliwia zdobycie doświadczenia w organizacji kodu źródłowego, podziale programu na klasy oraz pracy zespołowej z wykorzystaniem systemu kontroli wersji Git [9].

Aplikacja została podzielona na kilka plików źródłowych. Klasa odpowiedzialna za drzewo BST przechowuje jego strukturę i udostępnia operacje na elementach. Osobna część programu odpowiada za zapis i odczyt danych z plików. Główny plik programu zawiera menu konsolowe, dzięki któremu użytkownik może wybierać dostępne operacje.

1.2. Cel projektu

Głównym celem projektu jest stworzenie funkcjonalnego programu umożliwiającego praktyczne wykorzystanie binarnego drzewa poszukiwań [6]. Aplikacja ma prezentować sposób działania tej struktury danych oraz umożliwiać wykonywanie na niej operacji bez konieczności bezpośredniej ingerencji w kod źródłowy.

Do najważniejszych celów szczegółowych należą:

- zapoznanie się z zasadą działania binarnego drzewa poszukiwań [6],
- implementacja dynamicznej struktury danych w języku C++ [1],

- wykorzystanie wskaźników do łączenia poszczególnych węzłów [1],
- zastosowanie rekurencji podczas wykonywania operacji na drzewie [3],
- implementacja dodawania, usuwania i wyszukiwania elementów [5],
- umożliwienie wyświetlania elementów drzewa w różnych kolejnościach [5],
- przedstawienie struktury drzewa w postaci graficznej [5],
- zapis i odczyt danych z plików tekstowych oraz binarnych,
- podział aplikacji na klasy i osobne pliki źródłowe [2].

Projekt ma również na celu rozwijanie umiejętności programowania obiektowego [2]. Zastosowanie osobnych klas pozwala oddzielić odpowiedzialność za przechowywanie danych od operacji związanych z obsługą plików oraz interfejsem użytkownika.

1.3. Przeznaczenie aplikacji

Program przeznaczony jest przede wszystkim do celów edukacyjnych. Może być wykorzystywany jako przykład implementacji drzewa binarnego w języku C++ [1] oraz jako pomoc podczas nauki algorytmów i struktur danych [3, 4].

Użytkownik komunikuje się z aplikacją za pomocą menu tekstowego. Po uruchomieniu programu wyświetlana jest lista dostępnych operacji. Wybór odpowiedniej pozycji powoduje wykonanie przypisanej do niej funkcji.

Aplikacja umożliwia między innymi:

- dodawanie nowych wartości do drzewa [5],
- usuwanie wskazanych wartości [5],
- czyszczenie całej struktury [5],
- wyszukiwanie wartości i wyświetlanie drogi do niej [5],
- wyświetlanie elementów w kolejności preorder, inorder i postorder [5],
- graficzne przedstawienie drzewa [5],
- zapis danych do pliku tekstowego,
- zapis i odczyt drzewa w formacie binarnym,

- wczytywanie liczb z pliku tekstowego do drzewa.

Dzięki temu użytkownik może obserwować, jak zmienia się struktura drzewa po wykonaniu kolejnych operacji. Program pozwala również sprawdzić różnice pomiędzy poszczególnymi sposobami przechodzenia drzewa [3].

1.4. Binarne drzewo poszukiwań BST

Binarne drzewo poszukiwań, określane skrótem BST (ang. Binary Search Tree), jest strukturą danych, w której wartości przechowywane są w postaci hierarchicznie połączonych węzłów [3, 8]. Każdy węzeł może posiadać maksymalnie dwóch potomków: lewego i prawego [5].

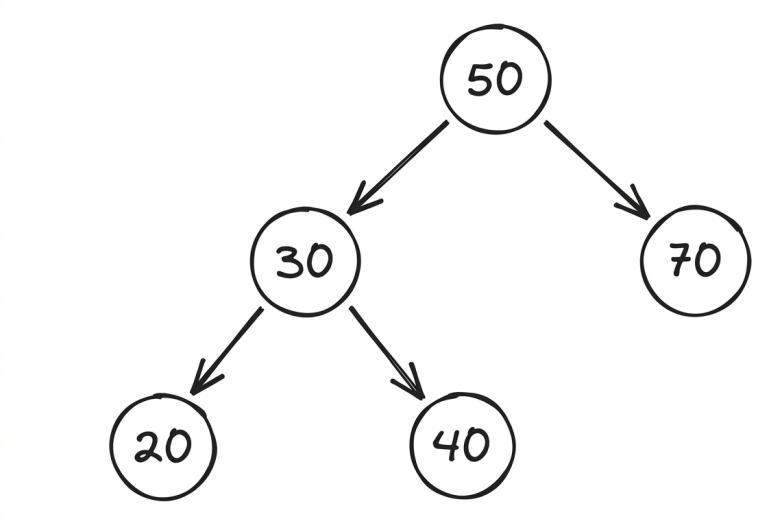
Najważniejszą cechą drzewa BST jest określona zależność pomiędzy wartościami znajdującymi się w poszczególnych częściach struktury [6]. Dla każdego węzła wartości znajdujące się w jego lewym poddrzewie są mniejsze od wartości tego węzła, natomiast wartości w prawym poddrzewie są większe [6].

Zasada ta obowiązuje rekurencyjnie dla każdego węzła drzewa [3]. Oznacza to, że zarówno lewe, jak i prawe poddrzewo również musi spełniać właściwości drzewa BST [6].

Podstawowe elementy drzewa to:

- **Korzeń (root)** – pierwszy, najwyżej położony węzeł drzewa [3]. Jest punktem początkowym wszystkich operacji.
- **Węzeł (node)** – pojedynczy element przechowujący wartość oraz wskaźniki do potomków [1, 5].
- **Lewe poddrzewo** – część drzewa zawierająca wartości mniejsze od wartości danego węzła [6].
- **Prawe poddrzewo** – część drzewa zawierająca wartości większe od wartości danego węzła [6].
- **Liść** – węzeł, który nie posiada potomków [5].
- **Krawędź** – połączenie pomiędzy węzłem a jednym z jego potomków [3].

Przykładowe drzewo BST może składać się z następujących wartości: 50, 30, 70, 20, 40, 60 oraz 80. Wartość 50 zostaje korzeniem. Liczby mniejsze od niej trafiają do lewej części drzewa, a większe do prawej [6].



Rys. 1.1. Przykładowa struktura binarnego drzewa poszukiwań BST.

1.4.1. Zasada działania drzewa BST

Dodawanie elementu do drzewa rozpoczyna się od korzenia [5]. Nowa wartość jest porównywana z wartością aktualnego węzła. Jeżeli jest mniejsza, algorytm przechodzi do lewego potomka. Jeżeli jest większa, przechodzi do prawego potomka [6].

Proces powtarza się do momentu znalezienia pustego miejsca [5]. Wartość zostaje wtedy umieszczona w nowym węźle. W ten sposób zachowana zostaje właściwość drzewa BST [6].

Wyszukiwanie elementu przebiega na podobnej zasadzie [3]. Algorytm porównuje szukaną wartość z aktualnym węzłem i na podstawie wyniku porównania wybiera lewe lub prawe poddrzewo [6]. Jeżeli wartość zostanie odnaleziona, operacja kończy się powodzeniem. Jeżeli algorytm dotrze do pustego wskaźnika, oznacza to, że poszukiwanego elementu nie ma w drzewie [1].

Usuwanie elementu jest bardziej złożone, ponieważ należy zachować prawidłową strukturę drzewa [3]. Wyróżnia się trzy podstawowe przypadki:

1. **Usuwanie liścia** – węzeł nie posiada potomków, dlatego można go bezpośrednio usunąć [5].

2. **Usuwanie węzła z jednym potomkiem** – w miejsce usuwanego węzła należy podłączyć jego jedyne poddrzewo [5].

3. **Usuwanie węzła z dwoma potomkami** – wartość węzła zastępuje się najmniejszą wartością z jego prawego poddrzewa, a następnie usuwa się węzeł zawierający tę wartość [5].

W implementacji drzewa BST istotną rolę odgrywa rekurencja [3]. Pozwala ona wykonywać operacje na kolejnych poziomach drzewa bez konieczności ręcznego zarządzania pozycją każdego węzła.

1.4.2. Właściwości i wydajność BST

Wydajność operacji wykonywanych na drzewie BST zależy przede wszystkim od jego wysokości [3]. Wysokość oznacza liczbę krawędzi na najdłuższej drodze prowadzącej od korzenia do liścia.

Jeżeli drzewo jest względnie zrównoważone, jego wysokość jest zbliżona do logarytmu liczby przechowywanych elementów [3]. W takim przypadku wyszukiwanie, dodawanie oraz usuwanie mogą być wykonywane w czasie logarytmicznym [3].

Jeżeli jednak elementy zostaną dodane w kolejności rosnącej lub malejącej, drzewo może przyjąć postać zbliżoną do listy [6]. Wówczas jego wysokość rośnie liniowo, a operacje mogą wymagać przejścia przez znaczną część struktury [3].

Tab. 1.1. Orientacyjna złożoność czasowa operacji w drzewie BST.

Operacja	**Średnio**	**Najgorzej**
Wyszukiwanie	$O(\log n)$	$O(n)$
Dodawanie	$O(\log n)$	$O(n)$
Usuwanie	$O(\log n)$	$O(n)$
Przejście całego drzewa	$O(n)$	$O(n)$

Warto zaznaczyć, że klasyczne drzewo BST nie wykonuje automatycznego równoważenia [3]. Istnieją również inne struktury, takie jak drzewa AVL lub czerwono-czarne, które stosują dodatkowe mechanizmy utrzymujące odpowiednią wysokość [3].

1.5. Sposoby przechodzenia drzewa

Przechodzenie drzewa polega na odwiedzaniu wszystkich jego węzłów w określonej kolejności [3]. Jest to jedna z podstawowych operacji wykonywanych na strukturach drzewiastych.

W projekcie wykorzystano trzy klasyczne sposoby przechodzenia drzewa binarnego [5]:

- preorder [5],

- inorder [5],
- postorder [5].

Każda z tych metod odwiedza wszystkie węzły, ale różni się kolejnością przetwarzania korzenia oraz jego poddrzew [3]. Wybór odpowiedniej metody zależy od celu wykonywanej operacji.

Do wyjaśnienia działania poszczególnych algorytmów można wykorzystać drzewo przedstawione na rysunku 1.1.

1.5.1. Preorder

Preorder, nazywany również przejściem w porządku przedrostkowym, polega na odwiedzaniu węzłów w następującej kolejności [3]:

1. odwiedzenie aktualnego węzła,
2. przejście do lewego poddrzewa,
3. przejście do prawego poddrzewa.

Zasada ta jest stosowana rekurencyjnie dla każdego poddrzewa [3]. Najpierw przetwarzany jest korzeń, a dopiero później jego potomkowie [5].

Dla przykładowego drzewa o wartościach 50, 30, 70, 20, 40, 60 i 80 wynik przejścia preorder będzie następujący [5]:

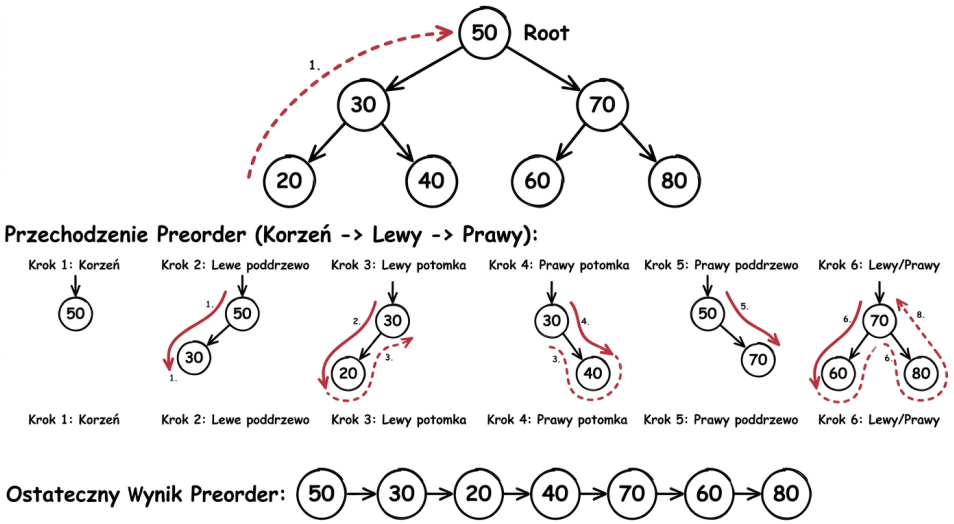
```
50, 30, 20, 40, 70, 60, 80
```

Metoda preorder jest przydatna między innymi podczas zapisywania struktury drzewa. Rozpoczyna przetwarzanie od korzenia, dzięki czemu może być wykorzystana do odtworzenia kolejności wstawiania elementów. Sam zapis preorder bez dodatkowych informacji nie zawsze wystarcza jednak do jednoznacznego odtworzenia dowolnego drzewa binarnego [3].

1.5.2. Inorder

Inorder, określany jako przejście symetryczne, wykonuje operacje w następującej kolejności [3]:

1. przejście do lewego poddrzewa,



Rys. 1.2. Przykład wykonania przejścia preorder w aplikacji.

2. odwiedzenie aktualnego węzła,
3. przejście do prawego poddrzewa.

W przypadku drzewa BST metoda inorder ma szczególne znaczenie [6]. Odwiedzanie najpierw lewego poddrzewa, następnie korzenia, a na końcu prawego poddrzewa powoduje wyświetlenie wartości w kolejności rosnącej [3, 6].

Dla przykładowego drzewa wynik będzie następujący [5]:

```
20, 30, 40, 50, 60, 70, 80
```

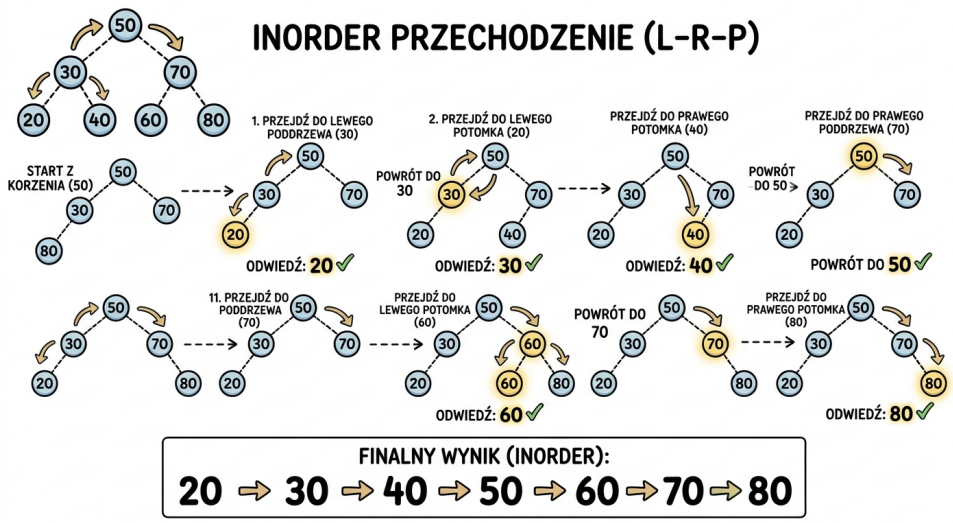
Dzięki tej właściwości inorder może być wykorzystywany do prezentowania wszystkich wartości przechowywanych w drzewie BST w uporządkowanej kolejności [5, 6].

Metoda ta jest również przydatna podczas sprawdzania poprawności struktury. Jeżeli drzewo spełnia zasady BST, wynik przejścia inorder powinien przedstawiać wartości w kolejności rosnącej [6].

1.5.3. Postorder

Postorder, nazywany przejściem w porządku przyrostkowym, polega na odwiedzaniu węzłów w następującej kolejności [3]:

1. przejście do lewego poddrzewa,
2. przejście do prawego poddrzewa,
3. odwiedzenie aktualnego węzła.



Rys. 1.3. Przykład wykonania przejścia inorder w aplikacji.

W tej metodzie korzeń jest przetwarzany dopiero po zakończeniu odwiedzania obu poddrzew [3]. Oznacza to, że węzły znajdujące się niżej w strukturze są odwiedzane przed swoimi przodkami [5].

Dla przykładowego drzewa wynik przejścia postorder będzie następujący [5]:

```
20, 40, 30, 60, 80, 70, 50
```

Postorder jest szczególnie przydatny podczas usuwania całej struktury drzewa [5]. Najpierw przetwarzane są potomne węzły, a dopiero później ich rodzice. Dzięki temu można bezpiecznie zwalniać pamięć zajmowaną przez poszczególne elementy, nie tracąc dostępu do ich potomków [1].

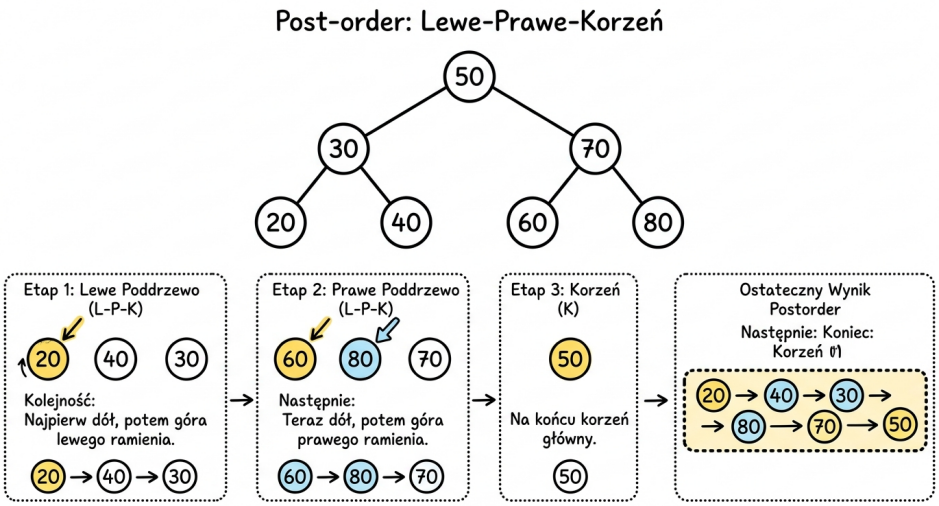
Metoda ta znajduje również zastosowanie w analizie wyrażeń zapisanych w postaci drzewa oraz w innych algorytmach przetwarzających struktury hierarchiczne [3].

1.5.4. Porównanie metod przechodzenia drzewa

Wszystkie trzy metody odwiedzają każdy węzeł dokładnie jeden raz, dlatego ich złożoność czasowa wynosi $O(n)$, gdzie n oznacza liczbę węzłów drzewa [3].

Różnica pomiędzy nimi dotyczy kolejności odwiedzania poszczególnych elementów [3].

Wybór odpowiedniej metody zależy od rodzaju zadania. Preorder pozwala rozpocząć przetwarzanie od korzenia, inorder umożliwia uzyskanie uporządkowanych wartości w drzewie BST, natomiast postorder jest przydatny między innymi podczas usuwania całej struktury [3, 5].



Rys. 1.4. Przykład wykonania przejścia postorder w aplikacji.

Tab. 1.2. Porównanie metod przechodzenia drzewa binarnego.

Metoda	**Kolejność**	**Wynik**
Preorder	Korzeń, lewo, prawo	50, 30, 20, 40, 70, 60, 80
Inorder	Lewo, korzeń, prawo	20, 30, 40, 50, 60, 70, 80
Postorder	Lewo, prawo, korzeń	20, 40, 30, 60, 80, 70, 50

1.6. Wykorzystanie kontenera STL std::vector

W projekcie wykorzystano kontener standardowej biblioteki C++ STL (*Standard Template Library*) o nazwie `std::vector` [12]. Jest to dynamiczna tablica umożliwiająca przechowywanie wielu elementów tego samego typu [1, 2]. Jej rozmiar może być automatycznie zwiększany lub zmniejszany w trakcie działania programu.

Kontener `std::vector` pozwala na wygodne przechowywanie danych bez konieczności ręcznego zarządzania pamięcią dla tablicy [1, 2]. Udostępnia również metody umożliwiające dodawanie, usuwanie oraz odczytywanie elementów [12].

W projekcie kontener ten może być wykorzystywany do gromadzenia wartości pobranych z drzewa BST, przechowywania wyników operacji oraz przekazywania danych pomiędzy poszczególnymi metodami programu.

Do najważniejszych operacji udostępnianych przez `std::vector` należą [12]:

- `push_back()` – dodawanie elementu na końcu kontenera;
- `pop_back()` – usuwanie ostatniego elementu;
- `size()` – zwracanie liczby przechowywanych elementów;

- `empty()` – sprawdzanie, czy kontener jest pusty;
- `clear()` – usuwanie wszystkich elementów;
- `at()` – dostęp do elementu z kontrolą zakresu indeksu.

Przykładowe wykorzystanie kontenera w języku C++ przedstawiono poniżej [2]:

```cpp
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> liczby;

    liczby.push_back(10);
    liczby.push_back(20);
    liczby.push_back(30);

    for (int liczba : liczby)
    {
        std::cout << liczba << std::endl;
    }

    return 0;
}
```

Listing 1. Przykładowe wykorzystanie kontenera std::vector

Zastosowanie kontenera `std::vector` upraszcza operacje na zbiorach danych oraz zwiększa czytelność kodu [2]. Dzięki wykorzystaniu gotowych mechanizmów biblioteki STL ograniczona zostaje konieczność samodzielnego implementowania dynamicznych tablic [1, 2].

Kontener `std::vector` stanowi przykład wykorzystania standardowej biblioteki języka C++ w projektowaniu i implementacji aplikacji [2, 12].

1.7. Wymagane technologie i narzędzia

Do realizacji projektu wykorzystano język C++ oraz narzędzia wspomagające tworzenie, testowanie i rozwijanie oprogramowania [1, 2]. Dobór technologii wynika z charakteru zadania, które wymaga implementacji dynamicznej struktury danych, obsługi plików oraz pracy zespołowej [3].

Tab. 1.3. Technologie i narzędzia wykorzystane w projekcie.

Technologia	**Zastosowanie**
C++	Główny język programowania wykorzystany do implementacji drzewa BST oraz pozostałych elementów aplikacji [1, 2].
Kompilator g++	Kompilowanie plików źródłowych i tworzenie pliku wykonywalnego programu.
Git	System kontroli wersji umożliwiający zapisywanie zmian w kodzie i zarządzanie historią projektu [9].
GitHub	Platforma do przechowywania repozytorium, współdzielenia kodu oraz pracy na osobnych gałęziach [10].
Visual Studio Code / IDE	Edycja kodu źródłowego, organizacja plików i uruchamianie programu.
LaTeX	Przygotowanie dokumentacji technicznej i opisowej projektu.

1.7.1. Język C++

C++ jest językiem programowania ogólnego przeznaczenia, który umożliwia tworzenie zarówno prostych aplikacji konsolowych, jak i rozbudowanych systemów [2]. W projekcie został wykorzystany ze względu na możliwość bezpośredniej pracy ze wskaźnikami, dynamicznym przydzielaniem pamięci oraz programowaniem obiektowym [1, 2].

Ważną rolę odgrywają wskaźniki, które umożliwiają łączenie węzłów drzewa [1, 5]. Każdy węzeł przechowuje wartość oraz odwołania do lewego i prawego potomka [5]. Dzięki temu struktura może być rozbudowywana dynamicznie, bez konieczności wcześniejszego określania jej rozmiaru [1].

1.7.2. Git i GitHub

Git jest rozproszonym systemem kontroli wersji [9]. Umożliwia rejestrowanie kolejnych zmian w plikach projektu, powrót do wcześniejszych wersji oraz równoległą pracę kilku osób [9].

GitHub jest platformą internetową, która umożliwia przechowywanie repozytoriów Git i współpracę nad kodem [10]. W projekcie wykorzystano repozytorium dostępne pod adresem:

`https://github.com/p-majchrzak/PZ_Projekt_Drzewo`

Narzędzia te wspierają organizację pracy zespołowej, umożliwiając oddzielne rozwijanie funkcjonalności i późniejsze łączenie zmian w głównej gałęzi projektu [9, 10].

1.8. Podsumowanie rozdziału

W rozdziale przedstawiono założenia projektu, jego cel oraz przeznaczenie. Omówiono również podstawy działania binarnego drzewa poszukiwań [5, 6], w tym budowę węzłów, zasady dodawania, wyszukiwania i usuwania elementów [5].

Szczególną uwagę poświęcono trzem metodom przechodzenia drzewa: preorder, inorder oraz postorder [5]. Każda z nich realizuje odwiedzanie węzłów w innej kolejności, co pozwala wykorzystywać je do różnych zadań [3].

Przedstawiono także technologie i narzędzia niezbędne do realizacji projektu [1, 2, 9, 10]. Stanowią one podstawę do dalszej analizy problemu, zaprojektowania struktury programu oraz omówienia jego implementacji w kolejnych rozdziałach.

2. Analiza problemu

2.1. Obszary zastosowania drzew BST

Drzewa binarnych poszukiwań znajdują zastosowanie w sytuacjach, w których konieczne jest przechowywanie danych w uporządkowanej strukturze oraz szybkie wykonywanie operacji wyszukiwania, dodawania i usuwania elementów [1, 3]. Ich główną zaletą jest możliwość organizowania danych w taki sposób, aby nie było konieczne każdorazowe przeglądanie całego zbioru [3].

Przykładowe obszary zastosowania tej struktury danych to:

- **Systemy informatyczne** – przechowywanie i wyszukiwanie rekordów na podstawie określonego klucza.
- **Bazy danych** – organizowanie danych w strukturach umożliwiających sprawne wyszukiwanie wartości [1].
- **Aplikacje administracyjne** – przechowywanie uporządkowanych identyfikatorów, numerów lub innych wartości liczbowych.
- **Algorytmy sortowania** – wykorzystanie właściwości drzewa do uzyskania uporządkowanego ciągu elementów [3].
- **Nauka programowania** – poznawanie struktur dynamicznych, wskaźników, rekurencji oraz sposobów zarządzania pamięcią [1, 2].

W przedstawionym projekcie drzewo BST zostało wykorzystane jako główna struktura przechowywania liczb całkowitych. Program umożliwia wykonywanie operacji na zbiorze danych bez konieczności korzystania z gotowych kontenerów realizujących podobne zadania [12].

2.2. Analiza problemu

Podstawowym problemem rozwiązywanym przez aplikację jest efektywne zarządzanie zbiorem liczb całkowitych. Dane muszą być przechowywane w uporządkowany sposób, a użytkownik powinien mieć możliwość ich modyfikowania i przeglądania.

W przypadku przechowywania danych w zwykłej tablicy lub liście wykonanie niektórych operacji może wymagać przeglądania wielu elementów. Zastosowanie drzewa BST pozwala ograniczyć liczbę porównań w typowych przypadkach, ponieważ kolejne decyzje podejmowane są na podstawie wartości aktualnego węzła [3, 5].

Podczas projektowania programu należało uwzględnić następujące problemy:

- prawidłowe rozmieszczanie nowych elementów w strukturze [5];
- odnajdywanie wskazanej wartości [5];
- usuwanie elementów bez naruszania poprawności drzewa [3, 5];
- zwalnianie pamięci zajmowanej przez dynamicznie utworzone węzły [1];
- zachowanie danych podczas zapisywania i odczytywania plików;
- obsługę sytuacji, w których użytkownik wykonuje operację na pustym drzewie;
- pomijanie wartości, które zostały już wcześniej dodane.

Istotnym założeniem projektu jest przechowywanie wyłącznie liczb całkowitych typu `int` [12]. Każdy element drzewa jest reprezentowany przez osobny obiekt węzła, a połączenia pomiędzy węzłami realizowane są za pomocą wskaźników [1].

2.3. Analiza sposobu działania programu

Aplikacja została zaprojektowana jako program konsolowy. Użytkownik komunikuje się z nią za pomocą menu tekstowego, w którym wybiera operację, którą chce wykonać.

Po uruchomieniu programu tworzona jest struktura drzewa. Następnie wyświetlane jest menu główne. Program oczekuje na wybór użytkownika, wykonuje odpowiednią operację i ponownie wyświetla dostępne możliwości.

Ogólny przebieg działania aplikacji przedstawiono poniżej:

1. Uruchomienie programu.
2. Utworzenie pustego drzewa BST.
3. Wyświetlenie menu głównego.
4. Pobranie wyboru użytkownika.
5. Sprawdzenie poprawności wybranej operacji.
6. Wykonanie wskazanego działania.
7. Wyświetlenie wyniku operacji.

8. Oczekiwanie na naciśnięcie klawisza Enter.

9. Powrót do menu głównego lub zakończenie programu.

Takie rozwiązanie pozwala na wielokrotne wykonywanie różnych operacji bez konieczności ponownego uruchamiania aplikacji. Dodatkowo oczekiwanie na naciśnięcie klawisza Enter umożliwia użytkownikowi zapoznanie się z wynikiem przed przejściem do kolejnego działania.

```
========== MENU BST ==========
1. Dodaj element
2. Usun element
3. Usun cale drzewo
4. Szukaj drogi do elementu
5. Przejdz drzewo
6. Wyswietl drzewo graficznie
7. Zapisz drzewo do pliku tekstowego
8. Zapisz drzewo do pliku binarnego
9. Wczytaj drzewo z pliku binarnego
10. Wczytaj liczby z pliku tekstowego
0. Zakoncz program
==============================
Wybierz opcje: 1
Podaj liczbe: 50
Dodano element (jesli nie byl duplikatem).

Nacisnij ENTER, aby powrocic do menu...

========== MENU BST ==========
1. Dodaj element
2. Usun element
3. Usun cale drzewo
4. Szukaj drogi do elementu
5. Przejdz drzewo
6. Wyswietl drzewo graficznie
7. Zapisz drzewo do pliku tekstowego
8. Zapisz drzewo do pliku binarnego
```

Rys. 2.1. Menu główne aplikacji konsolowej

2.4. Analiza poszczególnych operacji

W aplikacji zaimplementowano zestaw operacji umożliwiających kompleksowe zarządzanie strukturą danych.

2.4.1. Dodawanie elementu

Operacja dodawania rozpoczyna się od pobrania liczby podanej przez użytkownika. Następnie program sprawdza jej położenie względem kolejnych węzłów drzewa [5, 6].

Jeżeli drzewo jest puste, nowa wartość zostaje umieszczona w korzeniu. W przeciwnym przypadku program porównuje ją z wartościami znajdującymi się w kolejnych węzłach i przechodzi do odpowiedniej gałęzi. Proces trwa do momentu znalezienia wolnego miejsca [5].

W przypadku wykrycia wartości, która znajduje się już w drzewie, dodawanie zostaje pominięte. Zapobiega to przechowywaniu powtarzających się elementów.

2.4.2. Wyszukiwanie elementu

Wyszukiwanie polega na porównywaniu poszukiwanej wartości z kolejnymi węzłami drzewa [5]. Na podstawie wyniku porównania program wybiera dalszą gałąź przeszukiwania.

Jeżeli poszukiwana wartość zostanie odnaleziona, operacja kończy się powodzeniem. W przeciwnym przypadku, gdy program dotrze do pustego wskaźnika, oznacza to, że wskazany element nie istnieje w strukturze [5].

W projekcie dodatkowo uwzględniono możliwość wyświetlenia drogi prowadzącej od korzenia do odnalezionego elementu. Pozwala to prześledzić sposób działania algorytmu wyszukiwania [5].

2.4.3. Usuwanie elementu

Usuwanie jest bardziej złożoną operacją, ponieważ po usunięciu węzła należy zachować poprawność całej struktury [3, 5].

Podczas analizy tej operacji wyróżniono trzy możliwe sytuacje [5]:

1. Usuwany węzeł nie posiada potomków. W takim przypadku można go bezpośrednio usunąć.

2. Usuwany węzeł posiada jednego potomka. W jego miejsce należy podłączyć istniejącego potomka.

3. Usuwany węzeł posiada dwóch potomków. Należy zastąpić jego wartość odpowiednim elementem z prawej lub lewej części drzewa, a następnie usunąć węzeł, z którego pobrano zastępczą wartość.

Każdy z tych przypadków wymaga odpowiedniego zmodyfikowania połączeń pomiędzy węzłami. Nieprawidłowe wykonanie operacji mogłoby doprowadzić do utraty części danych lub uszkodzenia struktury drzewa [3].

2.4.4. Czyszczenie drzewa

Operacja czyszczenia usuwa wszystkie elementy znajdujące się w strukturze. Ponieważ węzły są tworzone dynamicznie, konieczne jest zwolnienie pamięci zajmowanej przez każdy z nich [1].

Po zakończeniu operacji drzewo powinno znajdować się w takim samym stanie jak bezpośrednio po uruchomieniu programu, czyli nie powinno zawierać żadnych elementów.

2.4.5. Wyświetlanie struktury

Aplikacja umożliwia przedstawienie aktualnej postaci drzewa w konsoli. Dzięki temu użytkownik może obserwować rozmieszczenie elementów oraz sprawdzać, jak kolejne operacje wpływają na strukturę.

Oprócz graficznego przedstawienia drzewa dostępne jest również wyświetlanie jego elementów w różnych kolejnościach. Szczegółowe zasady przechodzenia po strukturze zostały omówione w rozdziale pierwszym [3, 5].

2.5. Praktyczny przykład działania algorytmu

W celu przedstawienia sposobu działania programu przeanalizowano ręcznie proces tworzenia drzewa oraz wyszukiwania wybranej wartości [5].

Przyjmijmy, że użytkownik kolejno wprowadza następujące liczby:

$$50,\ 30,\ 70,\ 20,\ 40,\ 60,\ 80$$

Pierwsza liczba zostaje umieszczona w korzeniu drzewa. Każda kolejna wartość jest porównywana z elementami, które zostały już dodane [5].

1. Wartość 50 zostaje umieszczona w korzeniu.
2. Wartość 30 jest mniejsza od 50, dlatego trafia do lewego potomka.
3. Wartość 70 jest większa od 50, dlatego trafia do prawego potomka.

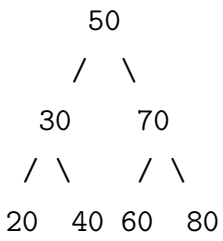
4. Wartość 20 jest mniejsza od 50, a następnie mniejsza od 30. Zostaje umieszczona po lewej stronie węzła 30.

5. Wartość 40 jest mniejsza od 50, ale większa od 30. Trafia na prawą stronę węzła 30.

6. Wartość 60 jest większa od 50, ale mniejsza od 70. Trafia na lewą stronę węzła 70.

7. Wartość 80 jest większa od 50 i 70. Trafia na prawą stronę węzła 70.

Po wykonaniu wszystkich operacji otrzymujemy następującą strukturę:



Rys. 2.2. Przykładowe drzewo utworzone na podstawie wprowadzonych danych

Następnie użytkownik chce odnaleźć wartość 60. Algorytm rozpoczyna wyszukiwanie od korzenia [5].

1. Porównanie z wartością 50: poszukiwana liczba jest większa, dlatego przechodzimy w prawo.

2. Porównanie z wartością 70: poszukiwana liczba jest mniejsza, dlatego przechodzimy w lewo.

3. Porównanie z wartością 60: wartości są równe, dlatego element zostaje odnaleziony.

Droga przebyta przez algorytm wygląda następująco:

$$50 \rightarrow 70 \rightarrow 60$$

W przedstawionym przykładzie odnalezienie wartości wymagało wykonania trzech porównań. Pokazuje to, w jaki sposób struktura drzewa ogranicza obszar przeszukiwania danych [3].

2.6. Analiza zapisu i odczytu danych

Ważnym elementem aplikacji jest możliwość zachowania utworzonego drzewa w pliku oraz odtworzenia go w późniejszym czasie.

W projekcie przewidziano dwa sposoby zapisu danych:

- **Format tekstowy TXT** – zapis wartości liczbowych w kolejności inorder. Plik może być odczytywany i edytowany za pomocą zwykłego edytora tekstu. Podczas odczytu wartości są ponownie dodawane do drzewa [5].
- **Format binarny BIN** – zapis danych w postaci binarnej wraz z informacją o strukturze węzłów. Pozwala to odtworzyć nie tylko przechowywane wartości, ale również pierwotny kształt drzewa.

Rozdzielenie tych dwóch sposobów zapisu wynika z ich odmiennego przeznaczenia. Format tekstowy zapewnia łatwy dostęp do samych danych, natomiast format binarny umożliwia dokładniejsze zachowanie struktury drzewa.

Program umożliwia również wczytywanie liczb z pliku TXT do pustego lub już istniejącego drzewa. W drugim przypadku odczytane wartości są dodawane do aktualnej struktury zgodnie z zasadami przyjętymi w projekcie [5].

2.7. Podsumowanie analizy

Przeprowadzona analiza pozwoliła określić sposób działania aplikacji oraz najważniejsze problemy, które należało rozwiązać podczas jej implementacji [1, 5]. Szczególną uwagę zwrócono na prawidłowe wykonywanie operacji na dynamicznej strukturze danych, zarządzanie pamięcią oraz możliwość zapisywania i odtwarzania informacji.

Przedstawiony przykład ręcznego tworzenia drzewa i wyszukiwania elementu pokazuje praktyczne wykorzystanie analizowanego algorytmu [5]. Na podstawie przeprowadzonych założeń możliwe było zaprojektowanie podziału programu na klasy odpowiedzialne za poszczególne zadania, co zostanie przedstawione w kolejnym rozdziale.

3. Projektowanie

3.1. Założenia projektowe

Projektowana aplikacja jest programem konsolowym napisanym w języku C++. Jej głównym zadaniem jest umożliwienie użytkownikowi wykonywania podstawowych operacji na binarnym drzewie poszukiwań (BST) [1, 3, 5].

Podczas projektowania przyjęto następujące założenia:

- program powinien posiadać czytelne menu tekstowe [12];
- użytkownik powinien mieć możliwość dodawania i usuwania elementów drzewa [3, 5];
- aplikacja powinna umożliwiać wyszukiwanie elementów [5];
- program powinien wyświetlać elementy drzewa w różnych porządkach [3, 5];
- użytkownik powinien mieć możliwość wyświetlenia struktury drzewa [5];
- aplikacja powinna obsługiwać zapis i odczyt danych z plików tekstowych oraz binarnych [5];
- kod powinien być podzielony na klasy i osobne pliki źródłowe [1];
- program powinien kontrolować poprawność danych wprowadzanych przez użytkownika.

Przyjęty podział na klasy pozwala oddzielić odpowiedzialność za przechowywanie danych, obsługę drzewa oraz operacje związane z plikami [1].

3.2. Struktura plików projektu

Projekt został podzielony na kilka plików, z których każdy odpowiada za określoną część aplikacji [1]. Takie rozwiązanie ułatwia zarządzanie kodem, jego analizę oraz późniejsze wprowadzanie zmian.

Struktura projektu przedstawia się następująco:

```
PZ_Projekt_Drzewo/
|
|-- Node.h
|-- Node.cpp
```

```
|
|-- BST.h
|-- BST.cpp
|
|-- FileManager.h
|-- FileManager.cpp
|
|-- main.cpp
|
|-- README.md
```

3.3. Charakterystyka plików

W projekcie zastosowano podział na pliki nagłówkowe oraz implementacyjne [1].

Tab. 3.1. Charakterystyka plików projektu

Plik	**Przeznaczenie**
Node.h	Deklaracja struktury węzła drzewa [1].
Node.cpp	Implementacja konstruktora węzła [1].
BST.h	Deklaracja klasy obsługującej drzewo BST [3, 5].
BST.cpp	Implementacja operacji na drzewie [3, 5].
FileManager.h	Deklaracja metod obsługi plików.
FileManager.cpp	Implementacja zapisu i odczytu danych [5].
main.cpp	Główna część programu i menu użytkownika [12].

3.4. Projekt klas

W aplikacji zastosowano trzy główne klasy: `Node`, `BST` oraz `FileManager` [1]. Dodatkowo plik `main.cpp` odpowiada za uruchomienie programu i komunikację z użytkownikiem [12].

3.4.1. Klasa Node

Klasa `Node` reprezentuje pojedynczy węzeł drzewa [1]. Każdy węzeł przechowuje wartość typu całkowitego oraz dwa wskaźniki prowadzące do lewego i prawego potomka [1].

Podstawowe pola klasy:

- `data` – przechowywana wartość [1, 12],
- `left` – wskaźnik do lewego potomka [1],
- `right` – wskaźnik do prawego potomka [1].

3.4.2. Klasa BST

Klasa `BST` odpowiada za zarządzanie strukturą drzewa [3, 5]. Przechowuje wskaźnik do korzenia i udostępnia metody umożliwiające wykonywanie operacji na drzewie [5].

Do najważniejszych operacji należą:

- dodawanie nowych elementów [5, 6],
- usuwanie wskazanych elementów [3, 5],
- wyszukiwanie wartości [5],
- czyszczenie całego drzewa [1],
- wyświetlanie elementów w porządkach preorder, inorder i postorder [3, 5],
- wyświetlanie struktury drzewa [5],
- wyszukiwanie drogi prowadzącej do wskazanego elementu [5].

3.4.3. Klasa FileManager

Klasa `FileManager` odpowiada za operacje związane z plikami. Jej zadaniem jest zapis oraz odczyt danych drzewa w dwóch formatach: tekstowym i binarnym [5].

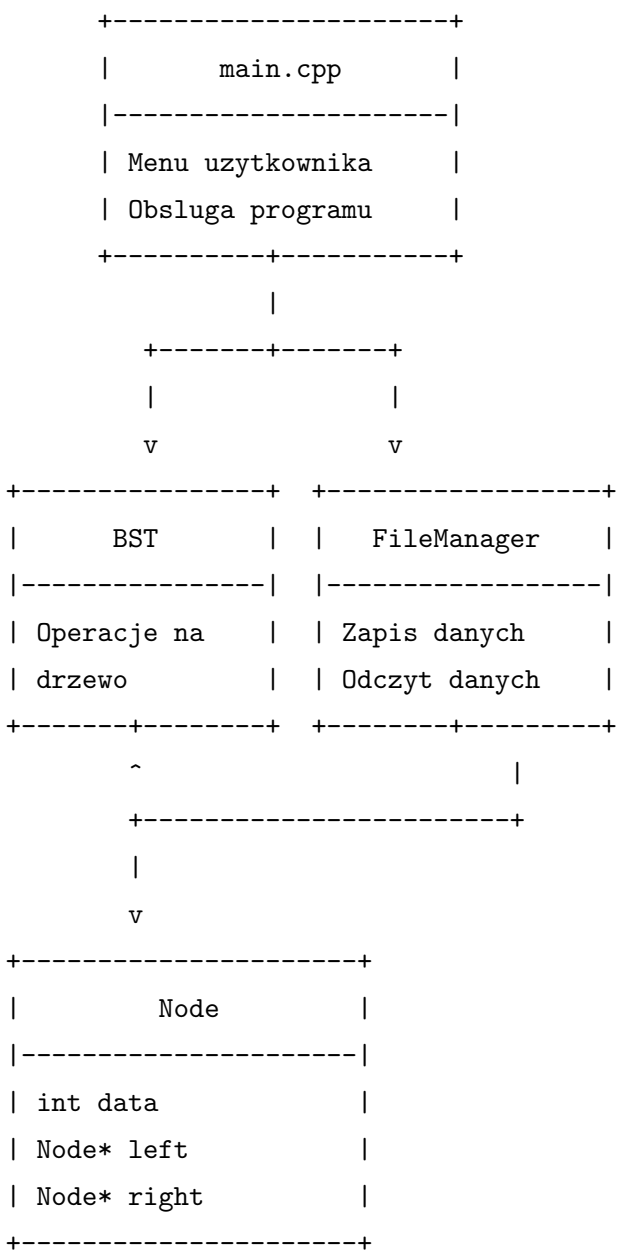
Oddzielenie obsługi plików od klasy `BST` pozwala zachować przejrzysty podział odpowiedzialności w projekcie [1].

3.4.4. Plik main.cpp

Plik `main.cpp` zawiera główną funkcję programu [12]. Odpowiada za utworzenie wymaganych obiektów, wyświetlanie menu, pobieranie wyboru użytkownika oraz wywoływanie odpowiednich metod klas [1, 12].

3.5. Diagram klas i zależności

Poniższy diagram przedstawia zależności pomiędzy głównymi elementami projektu [1].



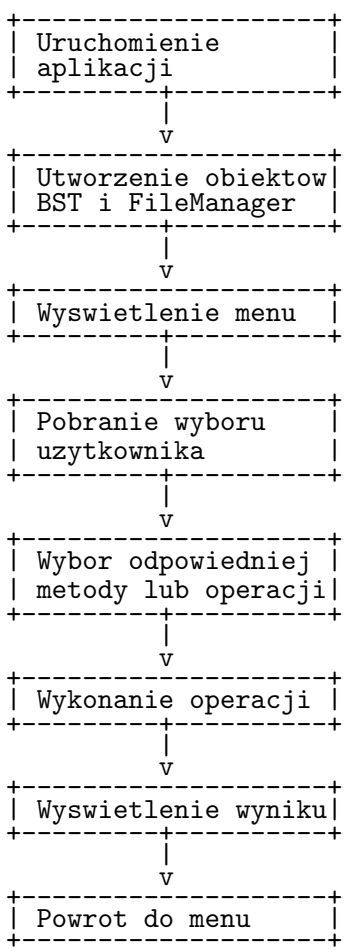
Rys. 3.1. Diagram klas i zależności w projekcie

3.6. Projekt przepływu działania aplikacji

Po uruchomieniu programu tworzone są obiekty odpowiedzialne za obsługę drzewa oraz operacje na plikach [1, 5]. Następnie wyświetlane jest menu, z którego użytkownik wybiera interesującą go operację [12].

Program pobiera wybór, sprawdza jego poprawność i wywołuje odpowiednią metodę [5, 12]. Po wykonaniu operacji prezentowany jest wynik, a następnie użytkownik może ponownie skorzystać z menu lub zakończyć działanie aplikacji [12].

Poniższy schemat przedstawia przepływ sterowania w aplikacji:



Rys. 3.2. Schemat przepływu sterowania w aplikacji

3.7. Zarządzanie pamięcią

Drzewo BST jest strukturą dynamiczną, co oznacza, że liczba jego węzłów może zmieniać się podczas działania programu [1, 3].

Każdy nowy węzeł jest tworzony dynamicznie [1]. Wskaźniki `left` i `right` pozwalają na połączenie go z pozostałymi elementami drzewa [1].

Podczas usuwania elementów należy odpowiednio zmodyfikować połączenia pomiędzy węzłami [3, 5]. W przypadku czyszczenia całego drzewa konieczne jest zwolnienie pamięci zajmowanej przez wszystkie węzły [1].

Odpowiednie zarządzanie pamięcią zapobiega powstawaniu wycieków pamięci i

pozwala na poprawne zakończenie działania programu [1].

3.8. Projekt obsługi plików

W aplikacji zaprojektowano dwa sposoby przechowywania danych: format tekstowy oraz binarny [5].

3.8.1. Format tekstowy TXT

W formacie tekstowym wartości drzewa zapisywane są jako liczby całkowite [5, 12]. Dane są zapisywane w kolejności inorder, czyli od najmniejszej do największej wartości [3, 5].

Plik tekstowy może być łatwo otwarty i edytowany przy użyciu zwykłego edytora tekstu [5].

3.8.2. Format binarny BIN

Format binarny służy do zapisu struktury drzewa w postaci danych binarnych [5]. W projekcie zastosowano sygnaturę `BST1` oraz znaczniki określające występowanie węzłów [5].

Takie rozwiązanie pozwala zachować nie tylko wartości elementów, ale również pierwotny układ drzewa [3, 5].

Tab. 3.2. Porównanie formatów zapisu danych

Cecha	**TXT**	**BIN**
Czytelność	Wysoka	Niska
Możliwość edycji	Tak	Nie w prosty sposób
Zachowanie struktury	Nie	Tak [5]
Zapis wartości	Liczby tekstowe [12]	Dane binarne [5]

3.9. Projekt procesu kompilacji

Projekt został przygotowany w języku C++ z wykorzystaniem standardu C++17 [12].

Do kompilacji można wykorzystać kompilator `g++`. Przykładowe polecenie kompilacji wszystkich plików projektu:

```
g++ -std=c++17 -Wall -Wextra -pedantic \
main.cpp Node.cpp BST.cpp FileManager.cpp \
```

`-o BST`

Zastosowane opcje oznaczają:

- `-std=c++17` – wykorzystanie standardu C++17 [12],
- `-Wall` – włączenie podstawowych ostrzeżeń,
- `-Wextra` – włączenie dodatkowych ostrzeżeń,
- `-pedantic` – kontrola zgodności ze standardem języka,
- `-o BST` – określenie nazwy pliku wykonywalnego.

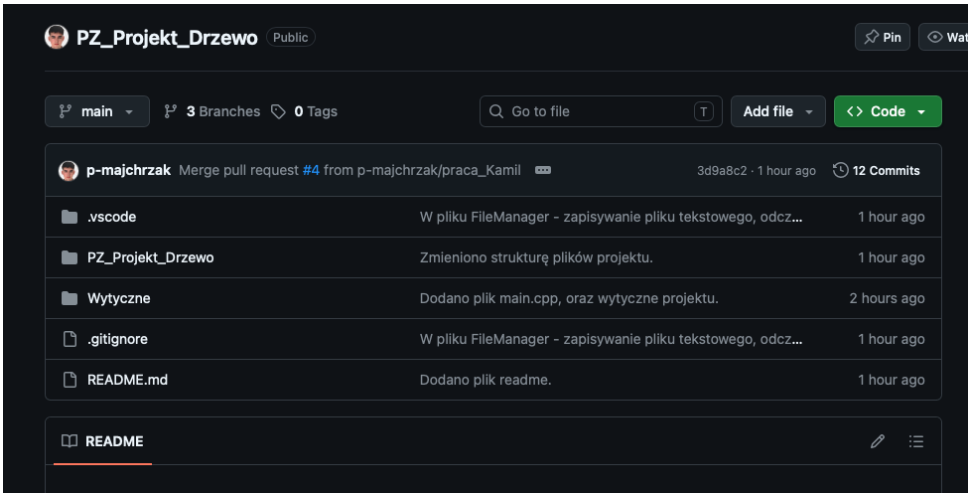
3.10. Organizacja pracy nad projektem

Kod źródłowy projektu jest przechowywany w repozytorium GitHub [10]. System kontroli wersji umożliwia śledzenie zmian wprowadzanych w kodzie oraz przechowywanie kolejnych wersji projektu [9, 10].

Repozytorium projektu:

https://github.com/p-majchrzak/PZ_Projekt_Drzewo

W projekcie wykorzystano system Git do zarządzania historią zmian oraz GitHub do przechowywania repozytorium zdalnego [9, 10].



Rys. 3.3. Repozytorium projektu w serwisie GitHub

3.11. Praca zespołowa z wykorzystaniem Git i GitHub

Projekt był realizowany przez dwuosobowy zespół. Do kontroli wersji wykorzystano system Git, natomiast GitHub służył do przechowywania repozytorium i współdzielenia kodu [9, 10].

Zgodnie z założeniami projektu każdy członek zespołu powinien:

- wykonać co najmniej 5 commitów [9, 10],
- utworzyć własną gałąź roboczą [10],
- pracować równolegle z drugim członkiem zespołu,
- uczestniczyć w scalaniu zmian [10],
- rozwiązać co najmniej 3 konflikty podczas scalania.

Łącznie wymagane jest zatem co najmniej 10 commitów oraz 6 konfliktów rozwiązanych przez członków zespołu.

3.11.1. Commity i gałęzie

Podczas pracy nad projektem zmiany były zapisywane w postaci commitów [9, 10]. Każdy commit zawiera określony zestaw modyfikacji oraz opis informujący o wykonanej pracy [9, 10].

Członkowie zespołu pracowali na osobnych gałęziach [10]. Pozwalało to rozwijać poszczególne elementy programu niezależnie od siebie, a następnie łączyć przygotowane zmiany z główną wersją projektu [10].

3.11.2. Scalanie zmian i rozwiązywanie konfliktów

Po zakończeniu pracy nad poszczególnymi elementami kodu zmiany były łączone z wykorzystaniem operacji merge [10].

W trakcie pracy zespołowej wymagane było wystąpienie co najmniej 6 konfliktów scalania. Każdy z dwóch członków zespołu powinien rozwiązać co najmniej 3 z nich.

Konflikty mogą wystąpić, gdy dwie osoby zmodyfikują ten sam fragment pliku w swoich gałęziach [10]. W takiej sytuacji konieczne jest przeanalizowanie obu wersji kodu i wybranie poprawnego rozwiązania [1].

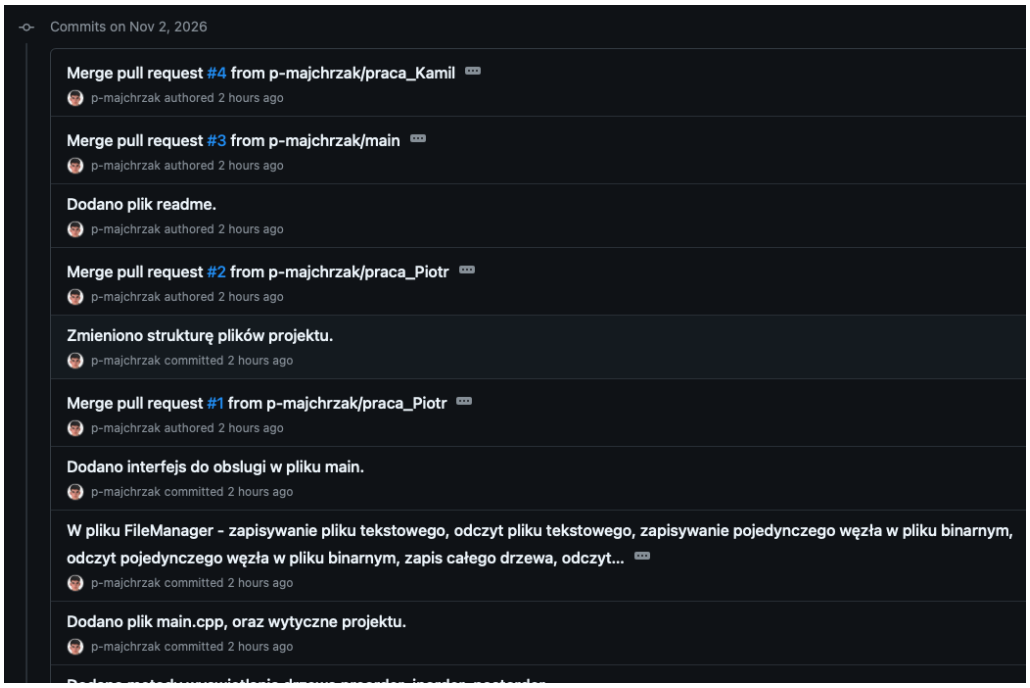
Po rozwiązaniu konfliktów należy sprawdzić poprawność połączonych plików [1]. Szczególnie istotne jest sprawdzenie, czy scalony kod kompiluje się i zachowuje wymagane funkcjonalności [1, 12].

3.11.3. Weryfikacja scalonego kodu

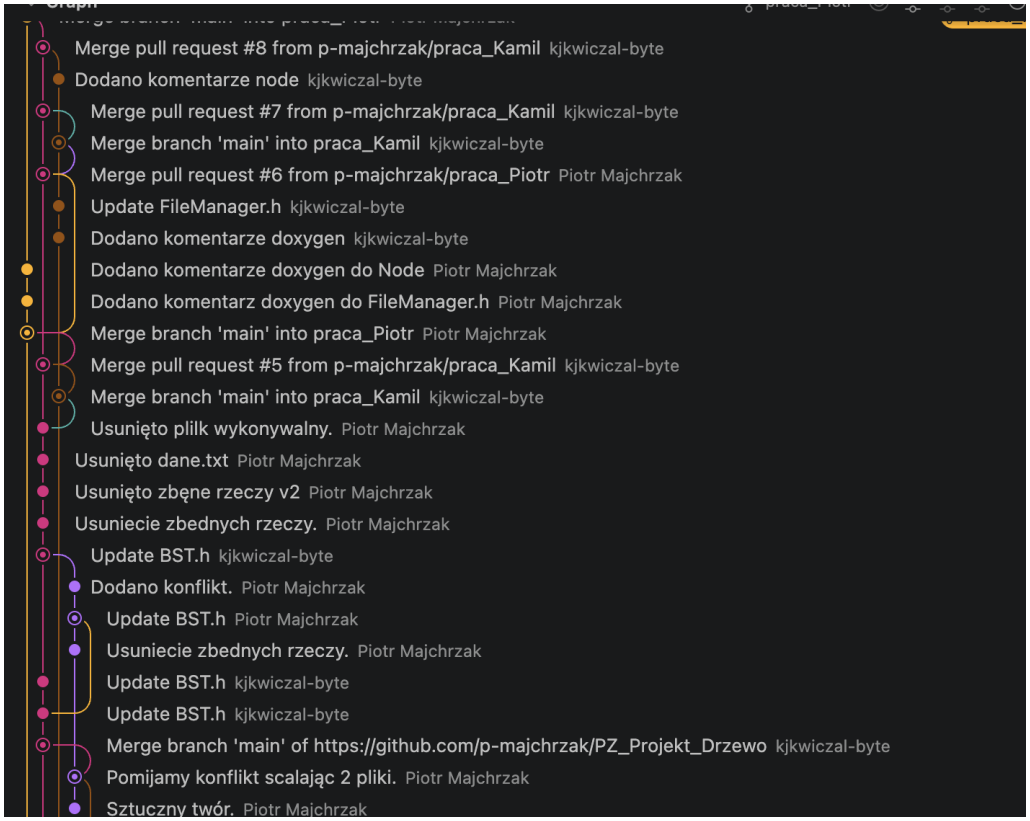
Po wykonaniu operacji merge sprawdzono pliki znajdujące się w głównej gałęzi repozytorium GitHub [10]. Celem było potwierdzenie, że zmiany obu członków zespołu zostały prawidłowo połączone [10].

3.11.4. Dokumentacja pracy w GitHub

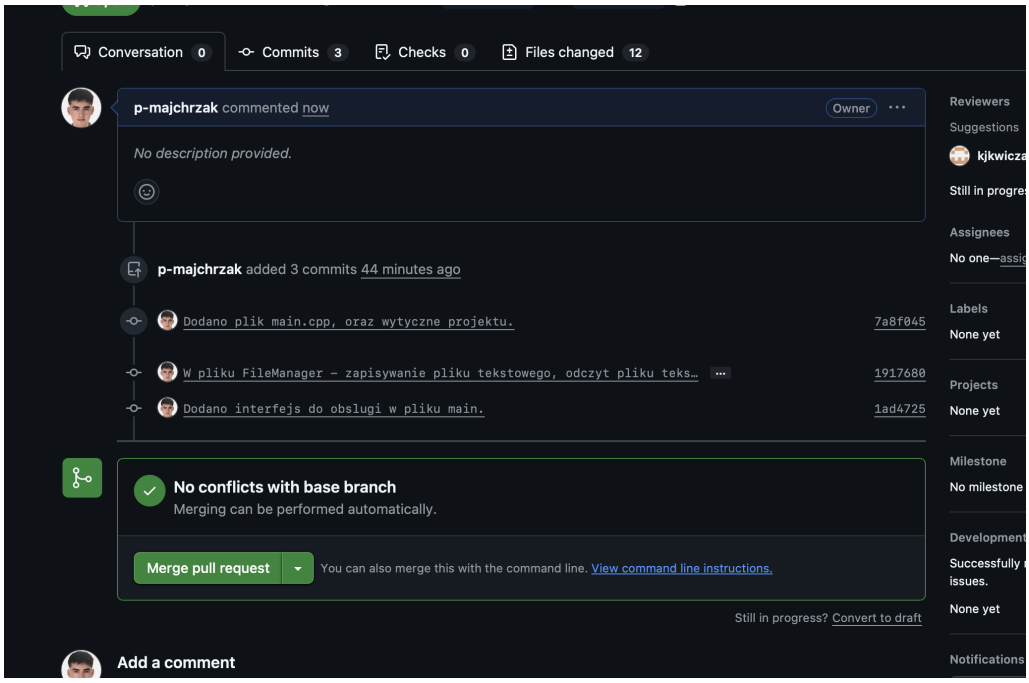
Poniżej przedstawiono osiem zrzutów dokumentujące pracę zespołu z wykorzystaniem GitHub [10].



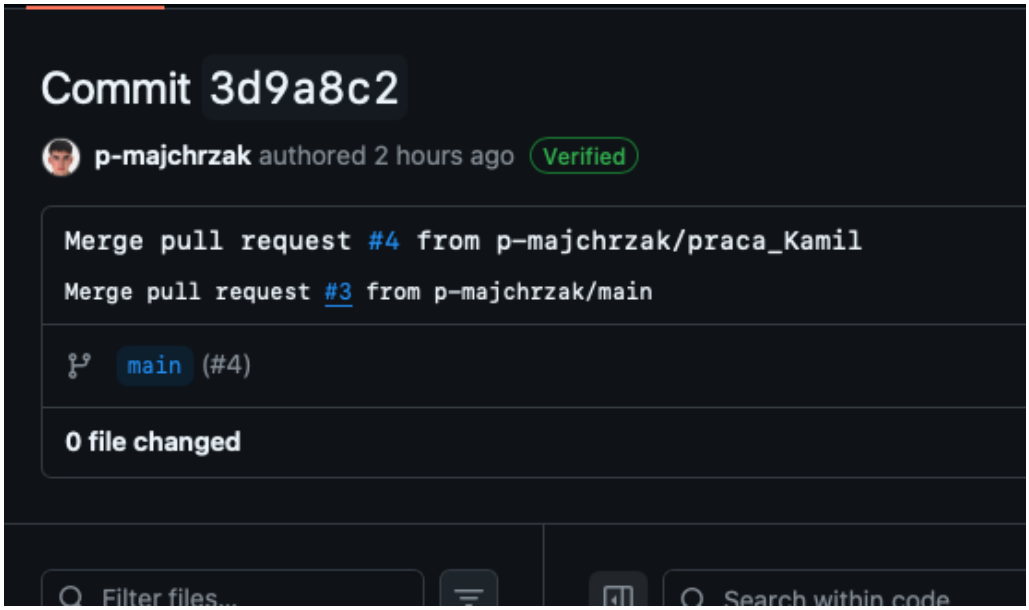
Rys. 3.4. Historia commitów w repozytorium GitHub



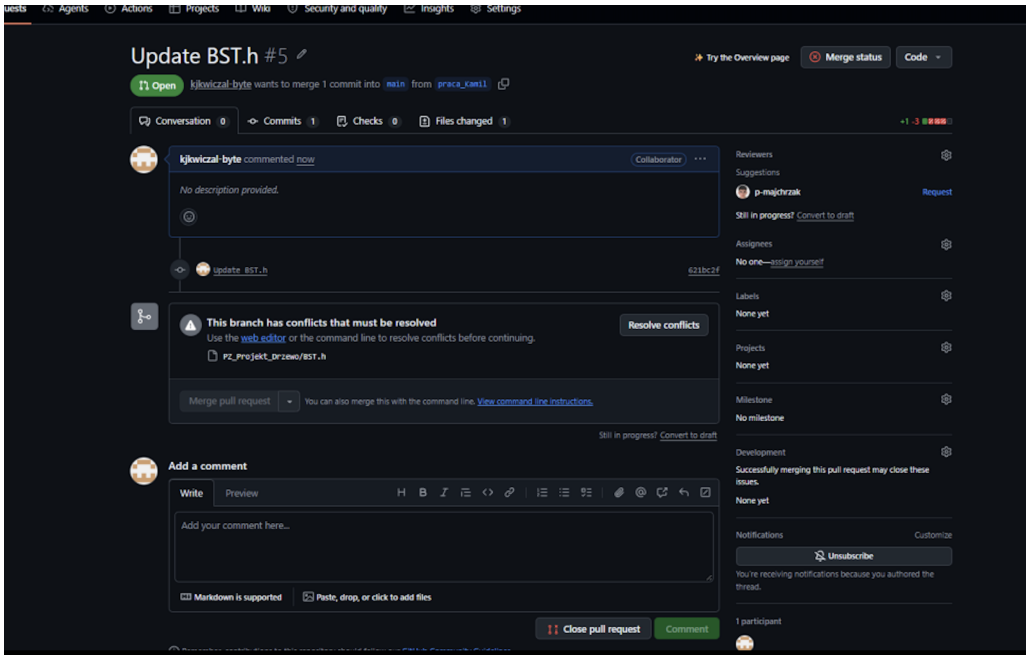
Rys. 3.5. Gałęzie robocze utworzone podczas realizacji projektu



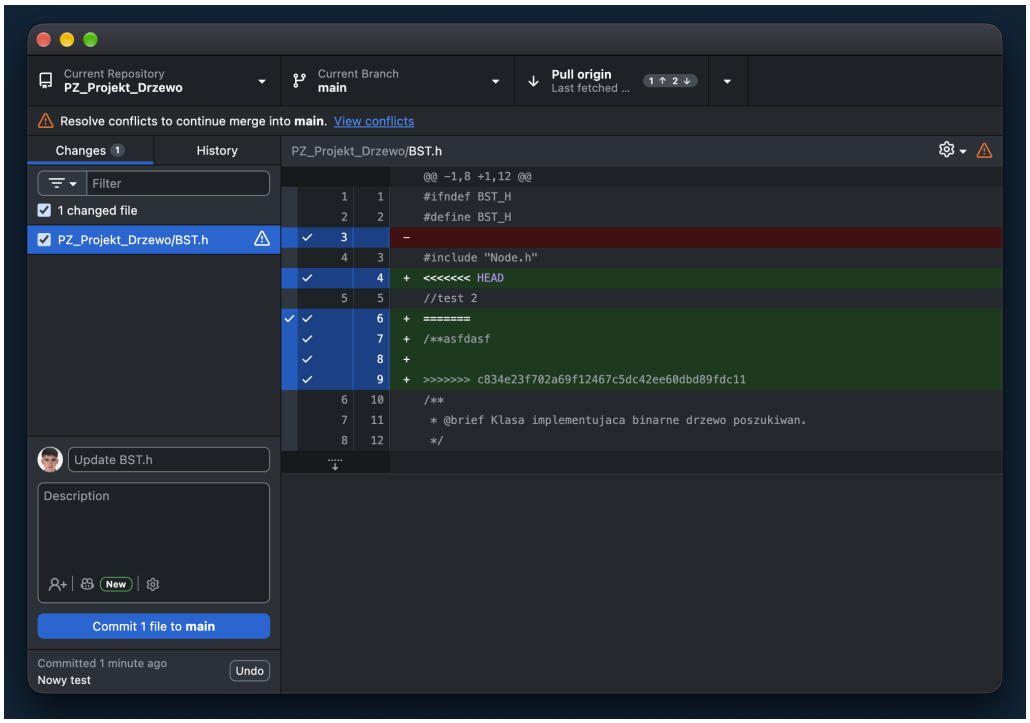
Rys. 3.6. Scalanie zmian z wykorzystaniem Pull Request



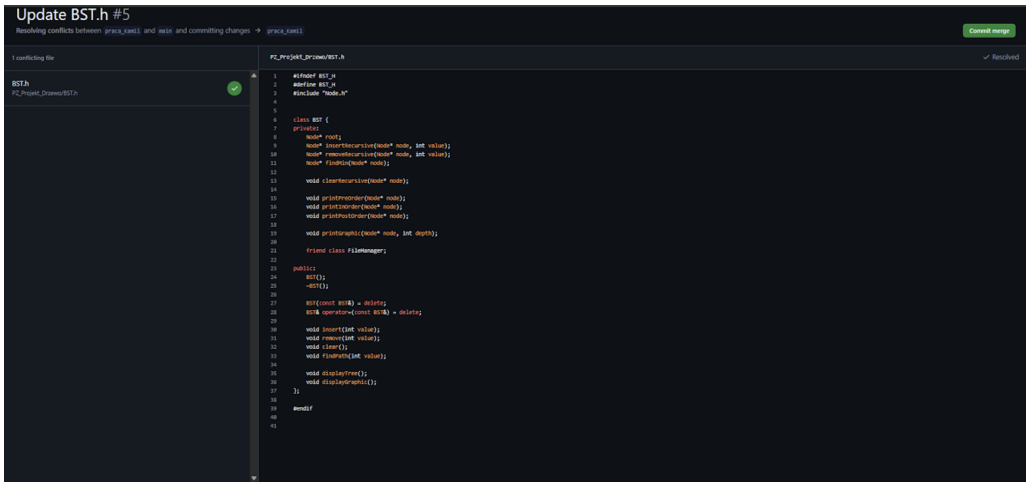
Rys. 3.7. Weryfikacja pliku po scaleniu zmian



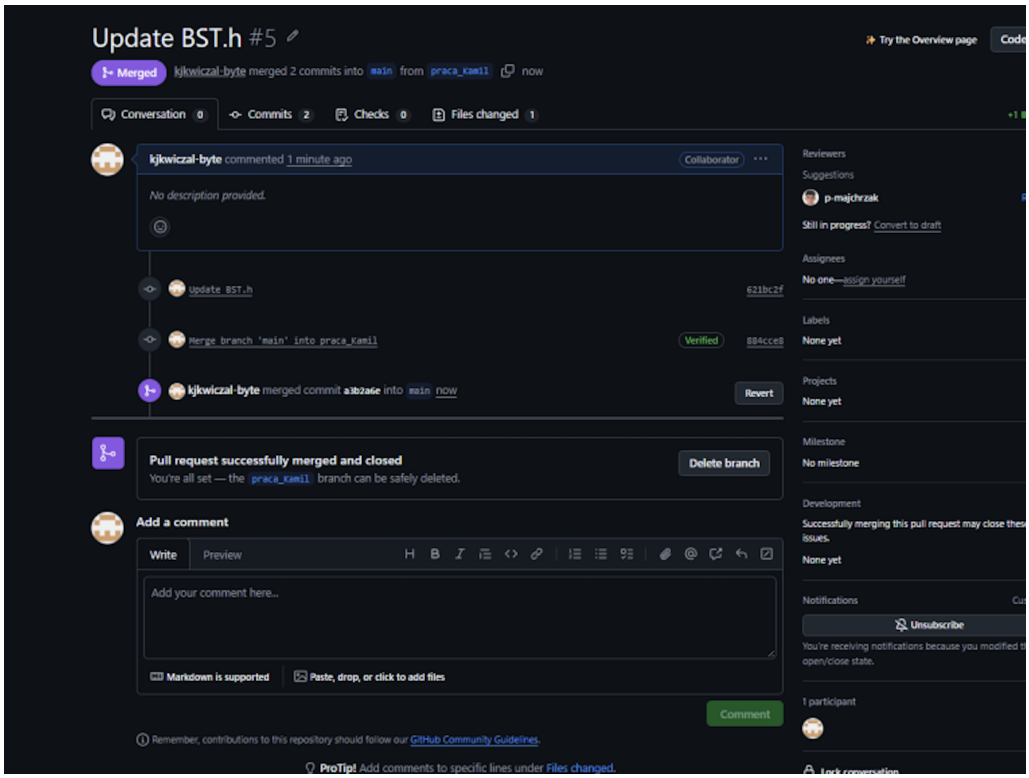
Rys. 3.8. Przykład konfliktu podczas scalania zmian



Rys. 3.9. Konflikt zmian przedstawiony w GitHub Desktop



Rys. 3.10. Rozwiązanie konfliktu podczas scalania zmian



Rys. 3.11. Sprawdzanie po scaleniu zmian

3.12. Podsumowanie projektowania

W rozdziale przedstawiono założenia projektowe aplikacji oraz podział programu na poszczególne pliki i klasy [1]. Opisano odpowiedzialność głównych elementów systemu, zależności pomiędzy nimi, przepływ działania aplikacji oraz sposób przechowywania danych [1, 3, 5].

Przyjęty projekt stanowi podstawę do implementacji poszczególnych funkcji programu [1, 12]. Dzięki podziałowi na niezależne klasy możliwe jest oddzielenie operacji

na drzewie od obsługi plików i interfejsu konsolowego [1, 3, 5, 12].

4. Implementacja

4.1. Implementacja struktury drzewa

Implementację projektu rozpoczęto od przygotowania klasy `Node`, która odpowiada za przechowywanie pojedynczego elementu drzewa [1]. Każdy węzeł przechowuje wartość typu całkowitego oraz dwa wskaźniki wskazujące na lewe i prawe poddrzewo [1].

Klasa `BST` zarządza całą strukturą drzewa [3, 5]. Przechowuje wskaźnik na korzeń i udostępnia metody umożliwiające wykonywanie operacji na elementach [5].

Zgodnie z założeniami projektu poszczególne klasy oraz funkcja główna programu zostały umieszczone w osobnych plikach źródłowych i nagłówkowych [1, 12].

Tab. 4.1. Podział implementacji na pliki

Plik	**Odpowiedzialność**
Node.h	Deklaracja struktury węzła drzewa [1].
Node.cpp	Implementacja konstruktora węzła [1].
BST.h	Deklaracja klasy zarządzającej drzewem [3, 5].
BST.cpp	Implementacja operacji na drzewie BST [3, 5].
FileManager.h	Deklaracja klasy obsługującej pliki.
FileManager.cpp	Implementacja zapisu i odczytu danych [5].
main.cpp	Menu konsolowe i obsługa wyboru użytkownika [12].

4.2. Implementacja operacji na drzewie BST

Główna część logiki programu została umieszczona w klasie `BST` [3, 5]. Zaimplementowano w niej operacje wymagane w założeniach projektu [3, 5].

4.2.1. Dodawanie elementów

Dodawanie nowego elementu rozpoczyna się od korzenia drzewa [5]. Wprowadzona wartość jest porównywana z wartością aktualnego węzła [5]. Jeżeli jest mniejsza, algorytm przechodzi do lewego poddrzewa, natomiast jeżeli jest większa, przechodzi do prawego poddrzewa [5].

Nowy węzeł zostaje utworzony w pierwszym napotkanym pustym miejscu [1, 5]. W przypadku wystąpienia wartości, która znajduje się już w drzewie, nie jest ona

dodawana ponownie [5].

```
Node* BST::insertRecursive(Node* node, int value) {
    if (node == nullptr) {
        return new Node(value);
    }

    if (value < node->data) {
        node->left = insertRecursive(node->left, value);
    }
    else if (value > node->data) {
        node->right = insertRecursive(node->right, value);
    }

    return node;
}
```

Listing 2. Przykładowy fragment dodawania elementu do drzewa

Powyższy fragment przedstawia zasadę działania rekurencyjnego dodawania elementów [5]. Właściwa metoda klasy rozpoczyna operację od korzenia drzewa [5].

4.2.2. Usuwanie elementów

Usuwanie elementów wymaga uwzględnienia trzech przypadków: usunięcia węzła bez potomków, węzła posiadającego jednego potomka oraz węzła posiadającego dwóch potomków [5].

W pierwszym przypadku węzeł może zostać bezpośrednio usunięty [5]. W drugim jego miejsce zajmuje istniejący potomek [5]. W przypadku dwóch potomków należy znaleźć odpowiednią wartość zastępującą usuwany element (najmniejszą wartość w prawym poddrzewie), a następnie usunąć jej pierwotny węzeł [5].

```
Node* BST::removeRecursive(Node* node, int value) {
    if (node == nullptr) {
        return nullptr;
    }

    if (value < node->data) {
        node->left = removeRecursive(node->left, value);
    }
    else if (value > node->data) {
        node->right = removeRecursive(node->right, value);
    }
    else {
        // Wezel bez lewego dziecka
```

```
14          if (node->left == nullptr) {
15              Node* temp = node->right;
16              delete node;
17              return temp;
18          }
19          // Wezel bez prawego dziecka
20          if (node->right == nullptr) {
21              Node* temp = node->left;
22              delete node;
23              return temp;
24          }
25
26          // Wezel posiada dwoje dzieci
27          Node* temp = findMin(node->right);
28          node->data = temp->data;
29          node->right = removeRecursive(node->right, temp->data);
30      }
31
32      return node;
33  }
```

Listing 3. Implementacja rekurencyjnego usuwania elementu z drzewa BST

Operacja jest wykonywana z zachowaniem właściwości drzewa BST, dzięki czemu po usunięciu elementu pozostałe wartości nadal znajdują się we właściwych poddrzewach [3, 5].

4.2.3. Czyszczenie drzewa

Program umożliwia również całkowite usunięcie wszystkich elementów drzewa [1]. Operacja ta jest realizowana rekurencyjnie [1]. Najpierw usuwane są potomkowie danego węzła, a następnie sam węzeł [1].

```
1  void BST::clearRecursive(Node* node) {
2      if (node != nullptr) {
3          clearRecursive(node->left);
4          clearRecursive(node->right);
5
6          delete node;
7      }
8  }
```

Listing 4. Przykładowa implementacja czyszczenia drzewa

Usuwanie rozpoczyna się od korzenia [1, 5]. Po zakończeniu operacji wskaźnik korzenia powinien wskazywać na `nullptr` [1]. Dzięki temu pamięć zajmowana przez

dynamicznie utworzone węzły zostaje zwolniona [1].

4.2.4. Wyszukiwanie drogi do elementu

Jednym z wymaganych elementów programu jest możliwość wyświetlenia drogi prowadzonej od korzenia do wskazanej wartości [5].

Algorytm rozpoczyna wyszukiwanie od korzenia i porównuje szukaną wartość z aktualnym węzłem [5]. W zależności od wyniku porównania przechodzi do lewego lub prawego potomka [5].

```
void BST::findPath(int value) {
    Node* current = root;

    std::cout << "Droga do elementu " << value << ": ";

    while (current != nullptr) {
        std::cout << current->data;

        if (current->data == value) {
            std::cout << " - Znaleziono!\n";
            return;
        }

        std::cout << " -> ";

        if (value < current->data) {
            current = current->left;
        }
        else {
            current = current->right;
        }
    }

    std::cout << "Brak elementu w drzewie.\n";
}
```

Listing 5. Wyświetlanie ścieżki do wybranego elementu w drzewie

Wszystkie odwiedzone wartości są prezentowane użytkownikowi [5, 12]. Jeżeli element nie istnieje w drzewie, program informuje o niepowodzeniu wyszukiwania [5].

4.2.5. Wyświetlanie drzewa

Aplikacja umożliwia graficzne przedstawienie struktury drzewa w konsoli [5, 12]. Węzły są wyświetlane z odpowiednim wcięciem zależnym od ich poziomu [5].

Dodatkowo użytkownik może wybrać jeden z trzech sposobów przechodzenia drzewa [3, 5]:

- preorder – najpierw odwiedzany jest korzeń, następnie lewe i prawe poddrzewo [3, 5],
- inorder – najpierw odwiedzane jest lewe poddrzewo, następnie korzeń i prawe poddrzewo [3, 5],
- postorder – najpierw odwiedzane są oba poddrzewa, a na końcu korzeń [3, 5].

W przypadku przechodzenia inorder wartości poprawnie zbudowanego drzewa BST są prezentowane w kolejności rosnącej [3, 5].

4.3. Implementacja obsługi plików

Obsługa zapisu i odczytu danych została umieszczona w osobnej klasie `FileManager` [5]. Rozdzielenie tej funkcjonalności od klasy `BST` pozwala oddzielić zarządzanie strukturą drzewa od operacji wykonywanych na plikach [1, 5].

4.3.1. Zapis i odczyt plików tekstowych

Pierwszym obsługiwanym formatem jest TXT [5]. Wartości drzewa są zapisywane w kolejności inorder, dzięki czemu w pliku znajdują się liczby uporządkowane rosnąco [3, 5].

Program umożliwia również wczytywanie liczb z pliku tekstowego [5]. Odczytane wartości mogą zostać dodane do pustego drzewa lub do drzewa, które zawiera już inne elementy [5].

W przypadku wczytywania do istniejącego drzewa nowe wartości są dodawane z wykorzystaniem standardowej operacji dodawania [5]. Obowiązuje przy tym zasada pomijania duplikatów [5].

4.3.2. Zapis i odczyt plików binarnych

Drugim obsługiwanym formatem jest BIN [5]. W przeciwieństwie do zapisu tekstowego format binarny umożliwia zachowanie pierwotnej struktury drzewa [5].

Plik rozpoczyna się od sygnatury `BST1` [5]. Następnie zapisywane są informacje o kolejnych węzłach oraz znaczniki określające występowanie pustych miejsc [5].

```
bool FileManager::writeNode(std::ostream& file, const Node* node) {
    std::uint8_t marker = (node == nullptr) ? 0 : 1;

    file.write(reinterpret_cast<const char*>(&marker), sizeof(
    marker));

    if (node == nullptr) {
        return file.good();
    }

    std::int32_t value = node->data;

    file.write(reinterpret_cast<const char*>(&value), sizeof(value)
    );

    return file.good() &&
           writeNode(file, node->left) &&
           writeNode(file, node->right);
}

bool FileManager::saveBinary(const BST& tree, const std::string&
    filename) {
    std::ofstream file(filename, std::ios::binary);

    if (!file) {
        return false;
    }

    const char signature[4] = {'B', 'S', 'T', '1'};

    file.write(signature, sizeof(signature));

    return file.good() &&
           writeNode(file, tree.root) &&
           file.good();
}
```

Listing 6. Zapis drzewa do pliku binarnego z wykorzystaniem nagłówka i znaczników węzłów

Podczas odczytu program odtwarza węzły oraz ich wzajemne połączenia [1, 5]. Dzięki temu możliwe jest przywrócenie nie tylko wartości elementów, ale również pierwotnego układu drzewa [3, 5].

Zastosowanie dwóch formatów pozwala na różne sposoby przechowywania danych [5]. TXT jest czytelny dla użytkownika, natomiast BIN umożliwia odtworzenie dokładniej struktury drzewa [5].

4.4. Implementacja menu programu

Funkcja `main()` odpowiada za uruchomienie aplikacji i komunikację z użytkownikiem [1, 12]. Po rozpoczęciu działania programu wyświetlane jest menu zawierające dostępne operacje [12].

```
int main() {
    BST tree;
    bool running = true;

    while (running) {
        showMenu();
        int choice = readInt("Wybierz opcje: ");

        switch (choice) {
            case 1: {
                int value = readInt("Podaj liczbe: ");
                tree.insert(value);
                std::cout << "Dodano element (jesli nie byl
   duplikatem).\n";
                break;
            }
            case 2: {
                int value = readInt("Podaj liczbe do usuniecia: ");
                tree.remove(value);
                std::cout << "Operacja usuwania zakonczona.\n";
                break;
            }
            case 3: {
                tree.clear();
                std::cout << "Drzewo zostalo wyczyszczone.\n";
                break;
            }
            // ... pozostale opcje menu ...
            case 0: {
                running = false;
                std::cout << "Zamykanie programu.\n";
                break;
            }
            default: {
```

```
34                  std::cout << "Nie ma takiej opcji.\n";
35                  break;
36              }
37          }
38      }
39      return 0;
40  }
```

Listing 7. Struktura pętli głównej i obsługa menu użytkownika w main.cpp

Użytkownik może wybrać między innymi dodawanie i usuwanie elementów, wyszukiwanie drogi do wskazanej wartości, czyszczenie drzewa, wyświetlanie jego struktury oraz obsługę plików [3, 5, 12].

Menu działa w pętli, dzięki czemu po wykonaniu jednej operacji można przejść do kolejnej bez ponownego uruchamiania aplikacji [12]. Wybranie opcji zakończenia powoduje opuszczenie pętli i zakończenie działania programu [12].

4.5. Testowanie programu i uzyskane wyniki

Po zakończeniu implementacji przeprowadzono testy sprawdzające poprawność działania poszczególnych funkcji [1, 3, 5].

Sprawdzeniu podlegały następujące operacje:

- dodawanie elementów do pustego drzewa [5],
- dodawanie elementów do drzewa zawierającego już dane [5],
- pomijanie powtarzających się wartości [5],
- usuwanie elementów w różnych przypadkach [3, 5],
- czyszczenie całego drzewa [1],
- wyszukiwanie istniejących i nieistniejących wartości [5],
- wyświetlanie drogi do wskazanego elementu [5],
- prezentowanie struktury drzewa [5],
- przechodzenie drzewa metodami preorder, inorder i postorder [3, 5],
- zapis i odczyt danych w formacie TXT [5],
- zapis i odczyt danych w formacie BIN [5],

- wczytywanie liczb z pliku TXT do pustego i istniejącego drzewa [5].

Do sprawdzenia działania programu można wykorzystać następujący zestaw wartości [5]:

```
50, 30, 70, 20, 40, 60, 80
```

Dla powyższych danych oczekiwane wyniki przechodzenia drzewa przedstawiono w tabeli [3, 5].

Tab. 4.2. Wyniki przechodzenia przykładowego drzewa

Metoda	**Wynik**
Preorder	50, 30, 20, 40, 70, 60, 80 [3, 5]
Inorder	20, 30, 40, 50, 60, 70, 80 [3, 5]
Postorder	20, 40, 30, 60, 80, 70, 50 [3, 5]

Przykładowo wyszukiwanie wartości 60 powinno prowadzić przez następujące węzły [5]:

```
50 -> 70 -> 60
```

Wyniki te pozwalają sprawdzić poprawność działania podstawowych operacji oraz zachowanie właściwości drzewa BST [3, 5].

4.6. Dokumentacja kodu za pomocą Doxygen

W projekcie wykorzystano narzędzie Doxygen do generowania dokumentacji kodu źródłowego [11]. Jego zadaniem jest automatyczne przygotowanie dokumentacji na podstawie komentarzy umieszczonych w plikach nagłówkowych i źródłowych [1, 11].

Komentarze dokumentacyjne opisują klasy, metody, parametry oraz zwracane wartości [1, 11]. Pozwala to łatwiej zrozumieć przeznaczenie poszczególnych elementów programu bez konieczności analizowania całej implementacji [1, 11].

Dokumentacja generowana przez Doxygen przedstawia strukturę klas oraz powiązania pomiędzy ich metodami [1, 11]. Ułatwia to analizowanie projektu i może być wykorzystywana podczas dalszego rozwijania aplikacji [1, 11].

4.7. Podsumowanie implementacji

W rozdziale przedstawiono realizację najważniejszych funkcjonalności programu zgodnie z przyjętymi założeniami [1, 3, 5]. Zaimplementowano strukturę drzewa BST oraz operacje dodawania, usuwania, czyszczenia, wyszukiwania drogi do elementu i prezentowania drzewa w konsoli [1, 3, 5].

Program umożliwia przechodzenie drzewa metodami preorder, inorder i postorder [3, 5]. Zastosowano również osobną klasę odpowiedzialną za zapis i odczyt danych w formatach TXT i BIN oraz import liczb z pliku tekstowego do pustego lub istniejącego drzewa [5].

Dodatkowo opisano sposób organizacji menu, generowania dokumentacji Doxygen oraz pracę zespołową z wykorzystaniem systemu Git i platformy GitHub [9, 10, 11, 12].

Przedstawione testy i zrzuty ekranu służą udokumentowaniu działania aplikacji oraz realizacji wymagań dotyczących współpracy zespołowej [9, 10].

5. Wnioski

Realizacja projektu pozwoliła na praktyczne wykorzystanie wiedzy dotyczącej struktur danych, algorytmów oraz programowania obiektowego w języku C++ [1, 3, 12]. Głównym celem było zaprojektowanie i wykonanie programu umożliwiającego zarządzanie binarnym drzewem poszukiwań (BST) [1, 3, 5].

W trakcie pracy zaimplementowano najważniejsze operacje związane z obsługą drzewa, takie jak dodawanie i usuwanie elementów, wyszukiwanie wartości, czyszczenie struktury oraz wyświetlanie drogi prowadzącej do wskazanego węzła [1, 3, 5]. Zastosowanie trzech metod przechodzenia drzewa: preorder, inorder i postorder pozwoliło zaprezentować jego zawartość na różne sposoby [3, 5].

Istotnym elementem projektu było również zastosowanie obsługi plików [5]. Możliwość zapisu i odczytu danych w formatach TXT oraz BIN rozszerzyła funkcjonalność programu [5]. Format tekstowy zapewnia czytelny zapis wartości, natomiast binarny pozwala zachować pierwotną strukturę drzewa [5]. Dodatkowo przygotowano możliwość wczytywania danych z pliku tekstowego zarówno do pustego, jak i istniejącego drzewa [5].

Podczas realizacji projektu wykorzystano podział programu na osobne klasy i pliki źródłowe [1, 12]. Takie rozwiązanie ułatwia organizację kodu, jego analizowanie oraz późniejsze wprowadzanie zmian [1]. Zastosowanie programowania obiektowego pozwoliło oddzielić zarządzanie strukturą drzewa od operacji związanych z obsługą plików [1, 5].

Przeprowadzone testy umożliwiły sprawdzenie poprawności działania poszczególnych operacji [1, 3, 5]. Szczególną uwagę zwrócono na przypadki usuwania węzłów posiadających różną liczbę potomków, pomijanie duplikatów oraz zachowanie właściwości drzewa BST po wykonaniu kolejnych operacji [3, 5].

Ważnym aspektem projektu była również praca zespołowa z wykorzystaniem systemu kontroli wersji Git i platformy GitHub [9, 10]. Tworzenie commitów, korzystanie z osobnych gałęzi oraz scalanie zmian pozwoliło poznać praktyczne aspekty wspólnego rozwijania oprogramowania [9, 10]. Rozwiązywanie konfliktów umożliwiło lepsze zrozumienie sposobu działania systemu kontroli wersji i znaczenia weryfikacji zmian przed ich połączeniem [1, 9, 10].

Dodatkowo wykorzystanie narzędzia Doxygen pozwoliło przygotować dokumentację kodu źródłowego [11]. Dzięki komentarzom dokumentacyjnym możliwe jest łatwiejsze zapoznanie się z przeznaczeniem klas, metod i poszczególnych elementów programu [1, 11].

Podsumowując, projekt pozwolił zrealizować założenia dotyczące implementacji binarnego drzewa poszukiwań oraz rozwinąć umiejętności związane z programowaniem w języku C++, zarządzaniem pamięcią dynamiczną, obsługą plików i pracą zespołową [1, 3, 5, 9, 10, 12]. Zdobyta wiedza może stanowić podstawę do realizacji bardziej rozbudowanych projektów wykorzystujących struktury danych i algorytmy [1, 3, 5].

Spis rysunków

1.1. Przykładowa struktura binarnego drzewa poszukiwań BST. 8
1.2. Przykład wykonania przejścia preorder w aplikacji. 11
1.3. Przykład wykonania przejścia inorder w aplikacji. 12
1.4. Przykład wykonania przejścia postorder w aplikacji. 13
2.1. Menu główne aplikacji konsolowej 19
2.2. Przykładowe drzewo utworzone na podstawie wprowadzonych danych 22
3.1. Diagram klas i zależności w projekcie 27
3.2. Schemat przepływu sterowania w aplikacji 28
3.3. Repozytorium projektu w serwisie GitHub 30
3.4. Historia commitów w repozytorium GitHub 33
3.5. Gałęzie robocze utworzone podczas realizacji projektu 33
3.6. Scalanie zmian z wykorzystaniem Pull Request 34
3.7. Weryfikacja pliku po scaleniu zmian 34
3.8. Przykład konfliktu podczas scalania zmian 35
3.9. Konflikt zmian przedstawiony w GitHub Desktop 35
3.10. Rozwiązanie konfliktu podczas scalania zmian 36
3.11. Sprawdzanie po scaleniu zmian 36

Spis tabel

1.1. Orientacyjna złożoność czasowa operacji w drzewie BST. 9
1.2. Porównanie metod przechodzenia drzewa binarnego. 13
1.3. Technologie i narzędzia wykorzystane w projekcie. 15
3.1. Charakterystyka plików projektu 25
3.2. Porównanie formatów zapisu danych 29
4.1. Podział implementacji na pliki 38
4.2. Wyniki przechodzenia przykładowego drzewa 46

Spis listingów

1. Przykładowe wykorzystanie kontenera std::vector 14
2. Przykładowy fragment dodawania elementu do drzewa 39
3. Implementacja rekurencyjnego usuwania elementu z drzewa BST . . . 39
4. Przykładowa implementacja czyszczenia drzewa 40
5. Wyświetlanie ścieżki do wybranego elementu w drzewie 41
6. Zapis drzewa do pliku binarnego z wykorzystaniem nagłówka i znaczników węzłów 43
7. Struktura pętli głównej i obsługa menu użytkownika w main.cpp . . . 44

Literatura

[1] Drozdek Adam, *C++. Algorytmy i struktury danych*, Helion, Gliwice.

[2] Horstmann Cay S., *Wprowadzenie do C++. Efektywne nauczanie*, Helion.

[3] Cormen Thomas H., Leiserson Charles E., Rivest Ronald L., Stein Clifford, *Wprowadzenie do algorytmów*, WNT, Warszawa 2007.

[4] Kurp Franciszek, *Algorytmy. Struktury danych i złożoność obliczeniowa*, Helion.

[5] Wałaszek Jerzy, *Drzewa BST*, https://eduinf.waw.pl/inf/alg/001_search/0107.php.

[6] Algorytm.org, *Drzewa poszukiwań binarnych*, https://www.algorytm.org/klasyczne/drzewa-poszukiwan-binarnych-bst.html.

[7] Zintegrowana Platforma Edukacyjna, *Drzewa binarne w C++*, https://zpe.gov.pl/a/przeczytaj/DhuqJvfMP.

[8] Wikibooks, *Drzewa wyszukiwań binarnych*, https://pl.wikibooks.org/wiki/Struktury_danych/Drzewa/Drzewa_wyszukiwa%C5%84_binarnych.

[9] Git Documentation, *Git Reference*, https://git-scm.com/docs.

[10] GitHub Docs, *About branches*, https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches.

[11] Doxygen, *Doxygen Manual*, https://www.doxygen.nl/manual/.

[12] cppreference.com, *C++ Reference*, https://en.cppreference.com/w/.